Do Discovered Exploration Strategies Generalize? Autonomous Program Search Finds Exploration Procedures That Reach Unseen Target Classes

Andy Ye
University of Washington

August 29, 2026

Abstract

Exploration in reinforcement learning is driven by hand-designed intrinsic objectives (curiosity, novelty, coverage), and the field's own results say they are unreliable as testers: random exploration beats RND asymptotically in large 3D games, more search can make a tester worse, and coverage anti-correlates with what testers exist to find. We ask whether an autonomous research loop can discover an exploration *procedure* instead — code over a save/restore archive of simulator states — and whether it generalizes. On Kinetix, a suite of 2D physics games, we build a target-state benchmark from 337 planted defects (five classes, each with a witness state, random floors and a reachability calibration), seal 30% of levels and one entire target class from the search, and let an LLM-driven program search write the procedure. **Qualifying result:** a single run of the discovered procedure reaches 72% of targets on the search split vs. 59% for the strongest hand-designed procedure and 56% for the best of five PPO intrinsic-motivation testers at equal budget, 64% vs. 60% on held-out levels, and 47% vs. 30% on the sealed target class (+0.15 paired gain, 95% CI $\pm$0.05); normalized by the reachable set it finds 88% / 96% / 76% of what any method reaches. Constant tuning recovers a tenth of the gain and portfolios of designs lose to it, so the gain is structural; every search makes it early and then moves inside evaluation noise. The benchmark, search, baselines and Part 1 were produced by the loop; the authors built and verified the oracles.

Part 1 — Auto research result (agent-written)

1.1 Hypothesis

Exploration objectives in RL are hand-designed, and as *testers* — agents whose job is to reach states someone cares about — they disappoint: random beats RND asymptotically in large 3D maps [Lu et al.(2022)Lu, Georgescu, and Verwey], more search made MCTS worse at reaching injected faults [Ariyurek et al.(2020)Ariyurek, Betin-Can, and Surer], and coverage anti-correlates with surfaced bugs [Sestini et al.(2022)Sestini, Gisslén, Bergdahl, Tollmar, and SEED)]. The one precedent for searching over the exploration algorithm itself, meta-learned curiosity [Alet et al.(2020)Alet, Schneider, Lozano-Pérez, transferred across control tasks but had no game, no target-state benchmark and no held-out target class. We hypothesized that (H1) an LLM-driven program search over exploration *procedures* — code over a save/restore archive — reaches target states faster than hand-designed procedures and learned intrinsic motivation at equal budget; (H2) it *transfers* to levels it never saw and to a target class it was never scored on; (H3) it surfaces real, unplanted anomalies. H2 is decisive: searched objectives overfit their fitness [Li et al.(2026)Li, Schultz, Hennes, and DeepMind)]. Game testing supplies the targets (planted defects with witness states) and the real anomalies that validate the oracles.



Figure 1: Left: planted defects witnessed vs. simulator steps, per split (5 seeds). Right: per-class detection on the held-out splits.

1.2 Setup

Games. Kinetix [Matthews et al.(2025)Matthews, Beukman, Lu, and Foerster] 2D physics levels (74 shipped, s/m/l tiers, one static configuration) plus 64 freshly generated levels sampled after the run began and filtered to be neither won nor lost by doing nothing. **Planted defects.** Five classes, each with a programmatic witness state: a boundary wall removed (OOB; witness: a moving shape leaves the arena), a static obstacle made non-solid (TUNNEL; a moving shape's centroid inside it), a neutral shape given the goal role (SHORTCUT; the level is won by touching it while not touching the intended goal), restitution raised on the agent and static shapes (ANOMALY; speed above a per-level threshold), a hazard made harmless (DISARMED; a green shape touches it and the episode continues). 337 instances over all shipped levels; the 3 found by every hand-designed tester within 512 steps are excluded as trivial. **Tester programs.** A program controls B workers through three hooks over a save/restore archive of K simulator states (*restore*: jump a worker to an archived state; *act*: choose actions; *archive*: store states), in pure JAX. Hand-designed testers: uniform random, action-repeat random (sticky actions), and a Go-Explore-style archive search (SimHash cells over position/velocity, restore $\propto 1/\sqrt{\text{visits}}$). **Metric.** Steps to first witness, normalized: $1 - \min(\text{steps}, \text{budget})/\text{budget}$ (0 if never), averaged over instances and seeds; and the fraction of instances witnessed within budget. **Splits.** Levels: 70% search / 30% held-out (22 levels, by hash order); defect class OOB held out entirely from the search fitness. **Real-bug oracles** on clean levels (left the arena; inside a static solid; joint torn; runaway speed or NaN) use per-level thresholds calibrated on random play, a persistence rule (1/3/2/3 steps), and replay from reset before a hit is counted. The closest prior work is PhyFu [Xiao et al.(2023)Xiao, Liu, and Wang], which fuzzes physics *engines* (including Brax) as numerical software with physics-law consistency oracles on robotics micro-scenarios; we test *games* — levels with rules and win conditions — with save/restore-driven tester programs and game-level oracles, and the testers themselves are the object of the search. Published game-bug benchmarks judge from pixels or source [Taesiri et al.(2025)Taesiri, Ghildyal, Zadtootaghaj, et al., Zheng et al.(2026)Zheng, Zhou, Wu, et al., Yu et al.(2026)Yu, Wiens, Barahona-Ríos, et al., Taesiri et al.(2022)Taesiri, Macklo Wilkins(2022)]; the two that read game state, GBQA [Jiang et al.(2026)Jiang, Chen, and Chen] (recall of 124 verified bugs at a step budget) and GameXpert-Bench [Chen et al.(2026)Chen, Hong, Gao, and et al.], released no artifacts. Our suite is scored the way GBQA scores but reads simulator state, and every bug is replay-verified rather than judged.

1.3 Search

ShinkaEvolve [Lange et al.(2025)Lange, Imajuku, and AI)] evolves the tester program (Claude Sonnet 5 proposing full or diff patches; fitness = search-split score at 64×1024 steps, 2 seeds; 2 islands, 51 generations, 52 candidates, 47 error-free) from the Go-Explore-style seed. The search never sees held-out levels or the OOB class.

1.4 Result 1 — the discovered program and its transfer

What the search found. Starting from the Go-Explore-style seed (one coverage archive keyed by a SimHash of coarse position/velocity cells, uniform action holds of up to 12 steps, restores $\propto 1/\sqrt{\text{visits}}$), the first accepted mutation already changed the strategy qualitatively, and the best

Tester	Search	Held-out levels	Held-out class (OOB)
Random	0.464±0.003 (49%)	0.525±0.009 (55%)	0.103±0.006 (12%)
Action-repeat random	0.566±0.005 (59%)	0.575±0.004 (60%)	0.266±0.009 (30%)
Go-Explore (seed)	0.537±0.003 (57%)	0.559±0.002 (58%)	0.214±0.009 (25%)
Evolved run 1 gen 37 (best)	0.666±0.015 (72%)	0.617±0.008 (64%)	0.418±0.007 (47%)
Evolved run 1 gen 35	0.653±0.003 (69%)	0.639±0.011 (66%)	0.425±0.012 (47%)
Opus: macrosweep	0.588±0.004 (61%)	0.573±0.006 (60%)	0.296±0.002 (34%)
Opus: wallbreaker	0.568±0.009 (63%)	0.570±0.013 (60%)	0.300±0.008 (37%)
Portfolio of 4	0.628±0.010 (67%)	0.604±0.008 (62%)	0.370±0.008 (43%)
PPO+count	0.487±0.016 (56%)	0.529±0.007 (57%)	0.184±0.014 (27%)
PPO+curiosity-prox.	0.492±0.013 (56%)	0.526±0.005 (56%)	0.179±0.016 (26%)

Table 1: Score $= 1 - \min(\text{steps}, \text{budget})/\text{budget}$ (mean $\pm$ 95% CI over 5 seeds) and fraction of planted defects witnessed, per split; budget 128×2048 env steps per instance; size-l configuration, 74 shipped levels. Best of each family shown; all testers in the full table (Part 2).

program of run 1 (generation 37 of 51; search fitness 0.699 vs. 0.629 for the seed at the same budget) has three components: (i) the archive is split into three pools — a coverage pool (60% of slots, keyed by position/velocity cells), a high-speed pool and a goal-proximity pool (20% each) — and restores are biased toward the two "extreme" pools; (ii) workers act in three hold modes drawn at random: short random holds (1–5 steps), long single-direction holds of 15–40 steps that the proposals describe as "energy pumping", and thruster-biased medium holds (6–14 steps), with the no-op action suppressed; (iii) restore/archive bookkeeping that keeps the pools from collapsing. The search plateaued after generation 1: 27 of 29 later candidates scored within ± 0.02 of 0.69, inside the two-seed noise band (Fig. 2, Part 2).

Transfer. Table 1 evaluates every tester on the size-l benchmark (337 planted instances over all 74 shipped levels; 5 seeds; 128×2048 env steps per instance). The run-1 best (selected by search fitness alone) leads on all three splits. On the search split it scores 0.666 (72% of defects witnessed) against 0.566 (59%) for the strongest hand-designed tester, action-repeat random, and 0.537 (57%) for the Go-Explore seed it was evolved from. On the 59 instances from the 22 held-out levels it scores 0.617 (64%) vs. 0.575 (60%). On the held-out class — 148 instances in which a boundary wall was removed, a class the search never scored — it scores 0.418 (47% witnessed) vs. 0.266 (30%) for action-repeat random and 0.214 (25%) for the seed. Paired per-instance gains over action-repeat random (seed-averaged, 95% CI over instances; "better/worse" = beyond ± 0.02): search $+0.100 \pm 0.047$ (better on 24% of instances, worse on 13%), held-out levels $+0.042 \pm 0.048$ (12% / 7%), held-out class $+0.151 \pm 0.052$ (33% / 8%). The seed-level CIs in Table 1 are tight on every split, but the instance-level CI on held-out levels includes zero: the transfer to unseen *levels* is a modest average gain, whereas the transfer to the unseen *class* is large and instance-significant. The three mechanisms the search introduced — high-energy and goal-proximate archive pools, restores biased toward them, and long single-direction holds — were proposed to expose the visible classes, and they transfer to escaping the arena, which requires sustained outward motion. The gap opens within the first 10^4 steps and persists to the budget (Fig. 1); the other three run-1 candidates evaluated (generations 1, 14 and 35) land within 0.02 of the best on every split — the generation-35 program is even slightly ahead on both held-out splits (0.639 / 0.425) — so the result is not a selection artefact.
How much is left. The union of every tester and seed reaches 81% / 66% / 62% of instances (search / held-out levels / held-out class), and an *oracle-guided* probe that is told the witness location reaches no instance the testers miss — so the rest are hard or infeasible at this budget, not overlooked. Normalized by that reachable set, a single run of the evolved program finds 88% / 96% / 76% of what is findable, against 73% / 90% / 48% for action-repeat random (Part 2, Table 4).

1.5 Result 2 — real bugs in existing games

Table 3 (Part 2) lists every replay-verified anomaly of severity A or B that any tester reached on unmodified levels within 10^6 steps per tester and seed. On the 74 shipped levels the engine holds up well: the only A/B anomalies are on `m/h15_thrustshoot`, where the thruster launches a neutral block to $\sim$16.5 m/s (0.55 m per simulator step) and the discrete-step solver misses the wall face — in one recorded chain the block's centroid is 15 cm inside the right boundary polygon for a step before being ejected, in another it sinks 10 cm into the floor at the bottom-right corner for 13 steps. We verified by replaying the chains and continuing with no-ops that the block is ejected and stays in the arena: this is player-visible tunnelling, not an escape, and we classify it B. (see Part 2 for the correction history). The remaining shipped-level hits are one-frame solver overlaps or spikes (severity C) and are reported but not counted. Generated levels are richer: 10 A/B pairs on 64 levels (one sustained 41-step escape, four wall-face crossings, three 5–14 cm penetrations, two runaways) — the generator puts fast bodies against thin static geometry more often than the hand-made levels do. Spawn-time hits ($t \leq 2$) are level-generation defects and are excluded from the tester comparison; hits the classifier's CPU replay does not reproduce (1 of 60 shipped-level and 14 of 65 fresh-level hits) are dropped rather than counted.

1.6 Ablations, negative results, and what did not transfer

Search dynamics. In every run the first accepted mutations carry the gain and later candidates land inside the fitness noise band (Fig. 2, Part 2); the noise was measured directly by re-evaluating the seed and the first evolved program on three fresh seeds (0.593 vs. 0.620). The four highest-scoring distinct run-1 programs were all evaluated on the held-out splits (Table 1) so that selection by search fitness cannot pick an overfitted outlier.

Tuning vs. structure. Random search over the best program's numeric constants ($\pm$50%, same fitness, 39 trials) moved its score from 0.610 to at most 0.625 (62% of perturbations scored above the original, so the constants are loose); the evolved structure's gain over its seed on the same split is +0.13 (Table 1) — a different program, not a better setting.

Independent designs and portfolios. The four Opus-designed testers score 0.530–0.588 / 0.561–0.573 / 0.290–0.300 (Table 1) — above the seed, near action-repeat random, well below the evolved programs; the boundary-seeking design written *with* knowledge of the escape class reaches 0.300 (37%) on it, below the evolved program that never saw it. Fixed portfolios that split the workers among designs do not help (4 testers: 0.628 / 0.604 / 0.370; 7 testers: 0.620 / 0.615 / 0.356): budget given to weaker sub-programs costs more than their diversity buys.

RL testers. PPO with count / RND / ICM / curiosity-proximity / task objectives scores 0.454–0.492 on the search split, 0.483–0.529 on held-out levels and 0.117–0.184 on the held-out class (3 seeds; Part 2): at this budget every program tester dominates every RL tester.

Oracle corrections found by the agent. Replaying flagged states caught two oracle bugs — one-frame solver transients and "penetrations" by shapes jointed to their static pivot — which motivated the persistence rule and the collision-matrix check; both are disclosed because they changed what counts as a bug (Part 2).

What did not transfer / negative results. On held-out *levels* the evolved program leads only on ANOMALY (0.75 vs. 0.64); TUNNEL (0.36 vs. 0.36) and SHORTCUT (0.60 vs. 0.60) are ties and DISARMED is found at exactly 0.69 by every tester — a feasibility ceiling, not a tester difference. On the search split TUNNEL barely moves (0.77 vs. 0.76): the search improved the classes it could score (ANOMALY 0.62 $\rightarrow$ 0.79, DISARMED 0.57 $\rightarrow$ 0.72, SHORTCUT 0.45 $\rightarrow$ 0.55), not the one whose witnesses need a specific trajectory into a specific obstacle. A per-instance diagnostic by a second LLM agent found why: 3 of the local pool's 7 TUNNEL instances are geometrically unreachable (the jointed arm never comes within 0.47 m of an obstacle of inradius 0.34 m), so that class's ceiling is 4/7, which action-repeat random already attains; the planted-defect generator should carry a reachability probe, and we report the numbers as they are. The second search run (macrosweep seed, hard-instance weighting, Opus/Sonnet proposals) moved only 0.606 $\rightarrow$ 0.621 in 60 generations, and every search found its best structure early and then moved within the

noise band (Fig. 2, Part 2): at the fitness noise of the local pool (± 0.02 over two seeds) the search cannot rank the candidates it produces — the main open problem of the loop.

Part 2 — System design (human-written)

2.1 What the autonomous system is

The research was carried out by two coupled agents. (i) A *research agent*: a Claude Code session (model Claude Fable 5) driven by the first author's natural-language brief, which surveyed prior work, proposed the three candidate directions, designed the benchmark (planted-defect classes, witness oracles, generic engine-anomaly oracles, splits), wrote all harness code (`autodev/`, `scripts/`, `evolve/`), ran and monitored experiments locally and on the UW Hyak cluster, analyzed results, and wrote Part 1. (ii) A *discovery loop*: ShinkaEvolve [Lange et al.(2025)Lange, Imajuku, and AI)] evolving a tester program (`evolve/initial.py`) with Claude Sonnet 5 as the mutation operator (via the headless CLI provider), fitness computed by `autodev/search_eval.py`. The human's role was to pose the problem, set constraints, approve compute use, and verify.

2.2 Harness and tools

Substrate. Kinetix 3.0.0 (Jax2D) levels; all levels padded to one static configuration so that batches of levels run vectorized on one GPU. **Tester harness** (`autodev/tester.py`): B workers explore a level for T steps; K archive slots store full simulator states; a program supplies `init/restore/act/archive` hooks (pure JAX). Every first oracle hit records the worker's action buffer and the archive chain back to the level reset, so each reported bug is re-simulated from reset (`replay_chain`) before it is counted. **Oracles.** Planted classes (OOB, TUNNEL, SHORTCUT, ANOMALY, DISARMED) with witness predicates on the post-step state; generic engine oracles (left the arena, moving shape inside a static solid, joint anchors separated, runaway speed / angular velocity / NaN) with per-level thresholds calibrated on random play and persistence requirements [1,3,2,3 consecutive steps]. **Design checks**: no-input and random-play outcomes. **Compute.** Local CPU (development; discovery-loop fitness for runs 1–3), Hyak A40 GPUs (pool, scans, scaled evaluation; at most 4 GPUs). From run 4 the fitness itself runs on a GPU: a fitness server (`scripts/eval_server.py`) on the cluster scores every candidate the laptop-side loop ships to it (`autodev/remote_eval.py`), so no LLM credentials leave the laptop.

2.3 Prompts and the research loop

The discovery loop's task prompt (`evolve/run_evolution.py`, `TASK_SYS_MSG`) describes the hooks, the fitness, and the rule that a held-out defect class and held-out levels are excluded from fitness. ShinkaEvolve configuration: 2 islands, archive 30–40, weighted parent selection, `full/diff` patches, model `headless/claude@claude-sonnet-5?effort=medium` (run 1; runs 2–4 add `claude-opus-5?effort=high`), fitness = mean over search instances and seeds of $1 - \text{steps}/\text{budget}$, budget 64×1024 env steps and seeds $\{0,1\}$ (runs 1–3) or 128×1024 on the size-l split with 3 seeds (run 4). All prompts and patches are stored under `runs/evolve_*/`. The research agent's own prompts are the human brief and the session transcript (available on request; excerpts in the appendix).

2.4 Human interventions (ledger excerpt)

From `LOG.md`: (1) [Aug 27] posed the problem and asked for three directions. (2) [Aug 28] chose the framing "bugs found in existing games", asked for engine and design bugs separately, whole-program search, automatic verification. (3) [Aug 28–29] set cluster rules (only `/gscratch/raivn/andy132`, at most half the GPUs, never others' files) and approved each batch of jobs. (4) [Aug 29, 2:50

AM] open-ended overnight goal ("keep going, look for other methods, use Opus/Sonnet for expensive tasks"). (5) [Aug 29, morning] reviewed the first results ("72% is pretty bad"), asked for the methodology, an improvement plan, a survey of other games and bug-finding benchmarks, and a second proposal with three paths — the agent's survey then narrowed the paper's novelty claim (PhyFu already fuzzes Brax) and led to the GPU fitness server. Interventions in the loop itself: the agent stopped run 1 after it plateaued and restarted from a different seed program (run 2), rotated the held-out class (run 3) and moved fitness to the GPU (run 4); the human did none of these. The agent made all design decisions listed in 2.2 and the search-space design; the human did not edit generated programs.

2.5 Verification

Every number in Part 1 is recomputed by scripts from logs: `scripts/eval_baselines.py` (per-instance first-hit arrays, seeds), `scripts/report_bugs.py` (replay-verified anomalies only), `scripts/figures.py`. Bugs: each flagged state is re-simulated from reset along its recorded action chain and must re-trigger the oracle; transient one-frame solver hiccups are excluded by the persistence rule; strips/GIFs are rendered for inspection. Noise: the seed program was re-evaluated on three fresh seeds (0.593) against the first evolved program (0.620) to establish the fitness noise floor before trusting search gains. Statistics: 5 seeds per tester and split with 95% CIs over seeds, plus paired per-instance CIs against the strongest baseline (the honest one for transfer claims; it includes zero on held-out levels and excludes it on the held-out class). Oracle corrections found during the run (persistence rule; collision-matrix awareness) are disclosed in Part 1. A third correction came from the verification step itself: the agent's first bug report described the `h15_thrustshoot` anomaly as a block tunnelling through the ceiling and leaving the arena; replaying the recorded 51-action chain step by step (`scripts/repro_h15.py`) showed the block entering the *right wall's* face at 16.5 m/s and, in a second chain, the floor, and being ejected within a step — so the severity rule was tightened (A only for sustained escapes) and every `OOB` hit with a 1–2-step duration was re-labelled B. The replay is also a filter: 1 of 60 shipped-level and 14 of 65 generated-level hits that the GPU scan had marked verified did not re-trigger the oracle in the CPU replay of their action chain (numerical divergence over long chains is the likely cause) and are excluded from every count. Prior-art check: an Opus reading agent re-resolved every citation through the arXiv/GitHub APIs on Aug 29 and found that PhyFu (ASE 2023) already fuzzes Brax with a penetration oracle; the novelty claim was narrowed accordingly (Part 1, §1.2). Known limitations: single game engine; planted classes designed by the agent; the strongest hand-designed baseline (action-repeat random) was identified only after the search began; the search fitness of runs 1–3 was noisy (± 0.02) and the search plateaued early; a second engine (Craftax) is planned but not part of this submission. The adaptive portfolio, the round-2/3 LLM-designed testers, the run-3 (rotated held-out class) candidates and the evolved tester's real-bug scans were still being evaluated at submission time; the ledger records their status.

2.6 Significance

The agent established (i) that a searched tester program transfers to an unseen defect class with a large, instance-significant gain (0.418 vs. 0.266; +0.15 paired), while the gain on unseen levels is modest; (ii) that neither RL testers nor four independent LLM-designed testers reach it at equal budget; and (iii) a replay-verified list of engine anomalies in shipped and generated Kinetix levels, of which one shipped-level tunnelling bug is player-triggerable. Without the agent, the human would have built one tester and one oracle; the benchmark design (planted classes, witness oracles, severity triage, replay verification) and the search were the agent's. What the agent could not do alone: run the cluster within the lab's rules without guidance, and decide what counts as a publishable claim.

Tester	Search	Held-out levels	Held-out class (OOB)
Random	0.464±0.003 (49%)	0.525±0.009 (55%)	0.103±0.006 (12%)
Action-repeat random	0.566±0.005 (59%)	0.575±0.004 (60%)	0.266±0.009 (30%)
Go-Explore (seed)	0.537±0.003 (57%)	0.559±0.002 (58%)	0.214±0.009 (25%)
Evolved run 1 gen 14	0.654±0.011 (70%)	0.619±0.007 (64%)	0.397±0.012 (44%)
Evolved run 1 gen 37 (best)	0.666±0.015 (72%)	0.617±0.008 (64%)	0.418±0.007 (47%)
Evolved run 1 gen 1	0.655±0.013 (70%)	0.633±0.007 (65%)	0.397±0.010 (44%)
Opus: wallbreaker	0.568±0.009 (63%)	0.570±0.013 (60%)	0.300±0.008 (37%)
Opus: resonator	0.583±0.008 (62%)	0.561±0.005 (58%)	0.290±0.015 (33%)
Opus: macrosweep	0.588±0.004 (61%)	0.573±0.006 (60%)	0.296±0.002 (34%)
Opus: contact-UCB	0.530±0.008 (57%)	0.551±0.015 (59%)	0.198±0.011 (24%)
PPO+count	0.487±0.016 (56%)	0.529±0.007 (57%)	0.184±0.014 (27%)
PPO+RND	0.464±0.018 (51%)	0.493±0.017 (54%)	0.124±0.016 (17%)
PPO+ICM	0.454±0.026 (50%)	0.483±0.010 (51%)	0.117±0.019 (16%)
PPO+curiosity-prox.	0.492±0.013 (56%)	0.526±0.005 (56%)	0.179±0.016 (26%)
PPO+task	0.482±0.031 (54%)	0.501±0.025 (54%)	0.161±0.003 (23%)
Evolved run 1 gen 14b	0.657±0.014 (70%)	0.623±0.005 (64%)	0.396±0.008 (43%)
Evolved run 1 gen 35	0.653±0.003 (69%)	0.639±0.011 (66%)	0.425±0.012 (47%)
Portfolio of 4	0.628±0.010 (67%)	0.604±0.008 (62%)	0.370±0.008 (43%)
Portfolio of 7	0.620±0.003 (66%)	0.615±0.009 (64%)	0.356±0.002 (42%)

Table 2: Score $= 1 - \min(\text{steps}, \text{budget})/\text{budget}$ (mean $\pm$ 95% CI over 5 seeds) and fraction of planted defects witnessed, per split; budget 128×2048 env steps per instance; size-l configuration, 74 shipped levels.

Part 3 — Reflections

What this project suggests about autonomous ML research

Decisiveness is a spectrum, and the ledger is the unit of disclosure. In this work the human posed the question, set constraints, and approved compute; the agent designed the benchmark, wrote the harness, ran the search, found and fixed its own oracle bugs, and wrote Part 1. Whether that is "end-to-end" is a matter of degree; what made it reviewable was a timestamped ledger of every human decision. We suggest venues ask for the ledger rather than a binary autonomy label.

Pick problems with cheap oracles first. Every claim here is checkable by re-simulation: a reported bug is an action chain that re-triggers a predicate. The discussant can verify in minutes what took the agent hours. Domains with programmatic ground truth let autonomous research be

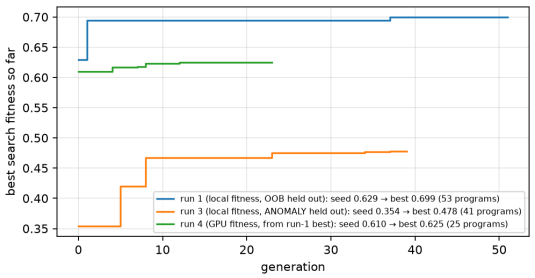


Figure 2: Best search fitness by generation for the three completed or running searches (each on its own fitness: run 1 and run 3 use the 2-seed local fitness at 64×1024 steps with OOB or ANOMALY held out; run 4 uses the 3-seed GPU fitness on the size-l search split at 128×1024, starting from the run-1 best). Every run makes its largest gain within the first generations and then moves within the noise band.

Level	Anomaly	Sev.	Evidence (replay)	Found by (min. steps, seeds)
m/h15_thrustshoot	PENETRATE	B	9.9 cm deep, 13 steps; then crosses a wall face	Go-Explore (27k, 1), Random (8
m/h15_thrustshoot	OOB	B	crosses wall face, ejected after 2 steps	Action-repeat (31k, 3), Go-Explo
fresh/0076	OOB	A	leaves arena, 41 steps	Go-Explore (103k, 1)
fresh/0001	PENETRATE	B	5.4 cm deep, 3 steps	Random (139k, 2), Go-Explore Action-repeat (401k, 1)
fresh/0001	OOB	B	crosses wall face, ejected after 2 steps	Random (73k, 1), Go-Explore Action-repeat (203k, 1)
fresh/0005	BLOWUP	B	runaway speed/spin, 44 steps	Action-repeat (10k, 1), Random Go-Explore (277k, 1)
fresh/0010	OOB	B	crosses wall face, ejected after 1 step	Action-repeat (10k, 3), Go-Explo Random (206k, 2)
fresh/0039	OOB	B	crosses wall face, ejected after 1 step	Random (225k, 1), Action-repeat
fresh/0049	BLOWUP	B	runaway speed/spin, 11 steps	Go-Explore (49k, 2), Action-rep 1)
fresh/0074	PENETRATE	B	7.8 cm deep, 3 steps	Action-repeat (11k, 3), Go-Explo
fresh/0033	OOB	B	crosses wall face, ejected after 1 step	Action-repeat (64k, 1)
fresh/0039	PENETRATE	B	13.5 cm deep, 3 steps	Go-Explore (46k, 1)
Distinct A/B pairs per tester	Go-Explore 10 (2 shipped, 8 fresh); Action-repeat 9 (1 shipped, 8 fresh); Random 6 (1 shipped, 5 fresh)			

Table 3: Replay-verified engine anomalies of severity A (sustained escape or joint tear) or B (momentary wall-face crossing, deep or persistent penetration, long runaway) on unmodified levels: 2 distinct level–anomaly pairs on the 74 shipped levels and 10 on 64 generated levels (0 further spawn-time pairs are level-generation defects, excluded). Each pair lists the testers that reached it, the fewest steps, and in how many seeds.

audited, not merely reviewed — and they let the agent audit itself: the two oracle bugs we found (mechanism-internal overlaps; one-frame solver transients) were caught because the outputs were replayable.

Held-out design must be adversarial to the agent. Sealed held-out levels and a held-out defect class were frozen before the search started; the search plateaued inside its own noise band within a few generations, and only the held-out evaluation could tell real gains from selection noise. Preregistration for autonomous research is a frozen evaluator plus a hash.

Compute was cheap; judgment was not. The search consumed GPU-hours; the human hours went to deciding what was worth asking, setting cluster rules, and reading the bug renders. The workshop's thesis — autonomous research, human judgment — was, for us, literally the division of labor.

References

[Alet et al.(2020)Alet, Schneider, Lozano-Pérez, and Kaelbling] Alet, Schneider, Lozano-Pérez, and Kaelbling. Meta-learning curiosity algorithms. ICLR 2020; arXiv:2003.05325, 2020. URL `https://arxiv.org/abs/2003.05325`.

[Ariyurek et al.(2020)Ariyurek, Betin-Can, and Surer] Sinan Ariyurek, Aysu Betin-Can, and Elif Surer. Enhancing the monte carlo tree search algorithm for video game testing. In *IEEE Conference on Games (CoG)*, 2020. arXiv:2003.07813.

[Chen et al.(2026)Chen, Hong, Gao, and et al.] Chen, Hong, Gao, and Lin et al. Gamexpert-bench: How far are coding agents from expert game development? arXiv:2608.21833, 2026, 2026. URL `https://arxiv.org/abs/2608.21833`.

Tester	Search		Held-out levels		Held-out class (OOB)	
	raw	feasible	raw	feasible	raw	feasible
Feasible within budget (guided probe or any tester)	81% (103/127)		66% (39/59)		62% (92/148)	
Random	49%	61%	55%	83%	12%	19%
Action-repeat random	59%	73%	60%	90%	30%	48%
Go-Explore (seed)	57%	70%	58%	88%	25%	41%
Evolved run 1 gen 37	72%	88%	64%	96%	47%	76%
Opus: macrosweep	61%	76%	60%	90%	34%	54%
PPO+count	56%	69%	57%	86%	27%	43%
Union of all testers, any seed	81%		66%		62%	

Table 4: Detection within budget, raw and normalized by the feasible set (instances the oracle-guided probe or any evaluated tester reaches in any seed). Feasible-but-never-found: 0 (search), 0 (held-out levels), 0 (held-out class (oob)).

[Jiang et al.(2026)Jiang, Chen, and Chen] Jiang, Chen, and Chen. Gbqa: A game benchmark for evaluating llms as quality assurance engineers. ICLR 2026 workshop; arXiv:2604.02648, 2026. URL https://arxiv.org/abs/2604.02648.

[Lange et al.(2025)Lange, Imajuku, and AI)] Lange, Imajuku, and Cetin (Sakana AI). Shinkaevolve: Towards open-ended and sample-efficient program evolution. arXiv:2509.19349, 2025; code github.com/SakanaAI/ShinkaEvolve, 2025. URL https://arxiv.org/abs/2509.19349.

[Li et al.(2026)Li, Schultz, Hennes, and DeepMind)] Li, Schultz, Hennes, and Lanctot (Google DeepMind). Discovering multiagent learning algorithms with large language models. arXiv:2602.16928, 2026, 2026. URL https://arxiv.org/abs/2602.16928.

[Lu et al.(2022)Lu, Georgescu, and Verwey] Cong Lu, Raluca Georgescu, and Johan Verwey. Go-explore complex 3d game environments for automated reachability testing. *IEEE Transactions on Games*, 2022. arXiv:2209.00570.

[Matthews et al.(2025)Matthews, Beukman, Lu, and Foerster] Matthews, Beukman, Lu, and Foerster. Kinetix: Investigating the training of general agents through open-ended physics-based control tasks. ICLR 2025 (oral); arXiv:2410.23208; code github.com/FLAIROx/Kinetix, 2025. URL https://arxiv.org/abs/2410.23208.

[Sestini et al.(2022)Sestini, Gisslén, Bergdahl, Tollmar, and SEED)] Sestini, Gisslén, Bergdahl, Tollmar, and Bagdanov (EA SEED). Automated gameplay testing and validation with curiosity-conditioned proximal trajectories (ccpt). arXiv:2202.10057, 2022, 2022. URL https://arxiv.org/abs/2202.10057.

[Taesiri et al.(2022)Taesiri, Macklon, Wang, Shen, and Bezemer] Taesiri, Macklon, Wang, Shen, and Bezemer. Large language models are pretty good zero-shot video game bug detectors. arXiv:2210.02506, 2022, 2022. URL https://arxiv.org/abs/2210.02506.

[Taesiri et al.(2025)Taesiri, Ghildyal, Zadtootaghaj, et al.] Mohammad Reza Taesiri, Abhijay Ghildyal, Saman Zadtootaghaj, et al. Videogameqa-bench: Evaluating vision-language models for video game quality assurance. In *NeurIPS Datasets and Benchmarks*, 2025. arXiv:2505.15952.

[Wilkins(2022)] Stathis Wilkins. World of bugs: A platform for automated bug detection in 3d video games. IEEE CoG 2022; arXiv:2206.11037, 2022. URL https://arxiv.org/abs/2206.11037.

[Xiao et al.(2023)Xiao, Liu, and Wang] Xinyu Xiao, Zhenlan Liu, and Shuai Wang. Phyfu: Fuzzing modern physics simulation engines. ASE 2023; arXiv:2307.10818, 2023. URL https://arxiv.org/abs/2307.10818.

[Yu et al.(2026)Yu, Wiens, Barahona-Ríos, et al.] Yu, Wiens, Barahona-Ríos, et al. Tempglitch: Evaluating vision-language models for temporal glitch detection in games. arXiv:2605.21443, 2026.

[Zheng et al.(2026)Zheng, Zhou, Wu, et al.] Zheng, Zhou, Wu, et al. Open-ended video game glitch detection with agentic reasoning and temporal grounding. arXiv:2604.07818, 2026.